

ICP133 - Fundamentos de Sistemas de Computação

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 9, 2026

Aula 5 - Precisão Finita

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 9, 2026

Números de Precisão Finita

- **No papel:** Usamos quantos dígitos forem necessários.
- **No Computador:** O espaço (memória) é limitado e definido no projeto do hardware.

Definição

Computadores lidam com uma quantidade **fixa** de dígitos. Isso nos força a trabalhar com um conjunto limitado de valores representáveis.

O que estudaremos nesta seção:

- Propriedades algébricas na precisão finita.
- Cálculo da quantidade de valores representáveis na base B .
- Representação de números inteiros com sinal.

Limitações da Precisão Finita

Considere um sistema hipotético que usa apenas **3 dígitos decimais** (000 a 999):

- **Valores impossíveis de representar:**
 - Maiores que 999 (Ex: 1000, 1500).
 - Menores que 0 (Ex: -1, -50).
 - Frações e números irracionais (Ex: 0,5 ou π).

Quebra de Propriedades

Diferente da matemática tradicional, os números de precisão finita **não possuem a propriedade de fechamento.**

A Propriedade de Fechamento

- **Na Matemática (Conjunto $\mathbb{Z}$):** Para qualquer i, j , os resultados de $i + j$, $i - j$ e $i \times j$ também são inteiros.
- **No Computador:** O resultado de uma operação pode "escapar" da faixa permitida.

Exemplos de Falha (Sistema de 3 dígitos)

- **Soma:** $900 + 500 = 1400$ (*Lupa: Cabe apenas 400?*)
- **Subtração:** $1 - 2 = -1$ (*Não há sinal negativo*)
- **Multiplicação:** $100 \times 10 = 1000$ (*Estourou o limite*)

OVERFLOW

- **O que é:** Ocorre quando o resultado de uma operação aritmética está fora da faixa de valores que o sistema consegue representar.
- **Analogia:** É como um hodômetro de carro que chega em 999.999km e volta para 000.000km.

Impacto no Software

Se o programador não prever o overflow, o computador pode produzir resultados bizarros (como somar dois números positivos e obter um número negativo ou um valor muito pequeno).

Propriedades Algébricas na Precisão Finita

Diferente da Matemática Clássica, onde os conjuntos numéricos são infinitos, o hardware precisa lidar com limites físicos. Isso afeta como as propriedades algébricas se comportam.

Detecção de Erro

No caso de **overflow**, o hardware (CPU) é projetado para identificar o estouro de capacidade e emitir um aviso (*flag*) para o sistema operacional.

Vamos analisar as três propriedades fundamentais:

1. **Comutatividade**
2. **Associatividade**
3. **Distributividade**

Comutatividade e Associatividade

- **Comutatividade:** Mantém-se inalterada.
 - $a + b = b + a$
 - $a \times b = b \times a$
 - A ordem dos operandos não altera o resultado, mesmo que ocorra overflow.
- **Associatividade:** É satisfeita, desde que o resultado final seja válido.
 - $(a + b) + c = a + (b + c)$
 - $(a \times b) \times c = a \times (b \times c)$

Associatividade

Se o resultado final estiver dentro da capacidade de representação, o valor será o mesmo. Se houver overflow, ele ocorrerá independentemente da ordem de execução, tornando o resultado inválido em ambos os casos.

Distributividade na Precisão Finita

A última propriedade fundamental é a **Distributividade** da multiplicação sobre a adição:

Fórmula

$$a \times (b + c) = (a \times b) + (a \times c)$$

- **No Computador:** A propriedade se mantém válida para números inteiros.
- **A Ressalva:** Assim como na associatividade, a igualdade é garantida se:
 1. O resultado final for **válido** (dentro da faixa representável).
 2. **OU** se ambos os caminhos de cálculo resultarem em um valor **inválido** (Overflow).

Conclusão das Propriedades

Para **Inteiros**, a Álgebra de Precisão Finita tenta mimetizar a Álgebra Clássica. O maior "vilão" não é a quebra das regras de ordem, mas sim o **estouro do limite de bits**.

Por que as Propriedades importam?

Mesmo sem decorar os nomes, usamos essas regras para simplificar cálculos no dia a dia:

- **Comutatividade + Associatividade:** Permitem somar uma lista de números em qualquer ordem (ex: agrupar números que somam 10 para facilitar a conta de cabeça).
- **Distributividade:** É a base para **colocar fatores em evidência** em expressões algébricas.

No Computador

Essas manipulações são usadas pelos **Compiladores** para otimizar o código, reorganizando as operações para que o processador as execute de forma mais rápida.

Quantos números cabem no sistema?

Para entender a capacidade de um hardware, precisamos resolver um problema de contagem:

O Problema

Dados B símbolos diferentes, de quantas maneiras podemos preencher n posições distintas (onde a ordem importa e símbolos podem se repetir)?

- **Posição 0:** B opções.
- **Posição 1:** B opções.
- ...
- **Posição $n - 1$:** B opções.

$$\text{Total de Combinações} = B^n$$

Princípio Multiplicativo: B^n

A fórmula B^n surge do princípio fundamental da contagem:

- Com **1 posição**: B possibilidades.
- Com **2 posições**: $B \times B = B^2$ possibilidades.
- Com **3 posições**: $B \times B \times B = B^3$ possibilidades.

Exemplo em Binário ($B = 2$)

Se um computador usa ****8 bits**** ($n = 8$):

$$2^8 = 256 \text{ combinações diferentes}$$

Se ele usa 10 bits:

$$2^{10} = 1.024 \text{ combinações}$$

Exercícios: Capacidade de Representação

Utilizando a fórmula B^n , determine a quantidade de combinações possíveis para cada caso:

- a) Quantos números diferentes podem ser expressos usando **2 dígitos** na base **10**?
- b) Quantos números diferentes podem ser expressos usando **5 dígitos** na base **2**?
- c) Quantos números diferentes podem ser expressos usando **32 dígitos** na base **2**?

Dica: Lembre-se da nossa tabela de potências de 2 vista anteriormente!

a) Base 10, $n = 2$:

$$10^2 = \mathbf{100} \text{ combinações}$$

(Os números de 00 a 99)

b) Base 2, $n = 5$:

$$2^5 = \mathbf{32} \text{ combinações}$$

(Os números binários de 00000 a 11111)

c) Base 2, $n = 32$:

$$2^{32} = \mathbf{4.294.967.296} \text{ combinações}$$

(Aproximadamente 4,3 bilhões de valores diferentes)

A quantidade de bits (n) define o "tamanho do universo" de números que podemos contar. Se o resultado de uma conta for o número $2^{32} + 1$, ele não "caberá" no universo que só temos 32 bits, gerando o **Overflow**.

Como representar o sinal?

Até agora, vimos números "sem sinal" (unsigned), onde todos os bits representam magnitude. Para incluir números negativos, precisamos de convenções:

- **Números sem sinal:** Faixa de 0 a $B^n - 1$.
- **Números com sinal:** Três esquemas principais:
 1. **Sinal-Magnitude:** O bit mais à esquerda indica o sinal ($0 = +, 1 = -$).
 2. **Excesso de 2^{n-1} :** O valor é deslocado por uma constante.
 3. **Complementos (1 e 2):** O bit de sinal faz parte do valor aritmético.

O problema do Zero

Como n bits geram uma quantidade par de combinações (2^n), ao usarmos uma para o zero, sobra um número ímpar de padrões para dividir entre positivos e negativos.

O Esquema Sinal-Magnitude

b_{n-1}	b_{n-2}	$\dots$	b_1	b_0
Sinal	Magnitude (Valor Absoluto)			

- **Bit de Sinal (b_{n-1}):** $0 \rightarrow$ Positivo (+)
- $1 \rightarrow$ Negativo (-).
- **Magnitude:** Representada pelos $n - 1$ bits restantes.
- **Faixa de Valores:** De $-(B^{n-1} - 1)$ até $B^{n-1} - 1$.

O Problema do "Zero Duplo"

Neste sistema, o zero possui duas representações:

- $+0$ (ex: 0000) e -0 (ex: 1000).

Isso desperdiça uma combinação e complica a lógica do hardware.

Representação Sinal-Magnitude ($n = 4$)

A tabela abaixo apresenta todas as 16 combinações possíveis para 4 bits:

Binário	Valor Decimal	Binário	Valor Decimal
0000	+0	1000	-0
0001	+1	1001	-1
0010	+2	1010	-2
0011	+3	1011	-3
0100	+4	1100	-4
0101	+5	1101	-5
0110	+6	1110	-6
0111	+7	1111	-7

- **Faixa de representação:** $[-7, +7]$.
- Observe a simetria: o bit mais significativo (MSB) apenas altera o sinal, mantendo a magnitude idêntica.

Exemplos e Exercício (Sinal-Magnitude)

Considere $n = 4$ bits:

- $0011_2 \rightarrow +3_{10}$
- $1011_2 \rightarrow -3_{10}$
- $1000_2 = 0000_2 = 0$

Converta os decimais para binário usando **5 bits** em **Sinal-Magnitude**:

- a) -15
- b) -1
- c) 0 (apresente as duas formas)
- d) 10

Gabarito: Exercício 21

Para $n = 5$ bits, o primeiro bit é o sinal e os outros 4 são a magnitude:

- a) $-15 \rightarrow$ Sinal: 1 $\rightarrow$ Magnitude (15_{10}): $1111_2 \rightarrow$ **11111**
- b) $-1 \rightarrow$ Sinal: 1 $\rightarrow$ Magnitude (1_{10}): $0001_2 \rightarrow$ **10001**
- c) $0 \rightarrow$ **00000** ($+0$) e **10000** (-0)
- d) $10 \rightarrow$ Sinal: 0 $\rightarrow$ Magnitude (10_{10}): $1010_2 \rightarrow$ **01010**

Reflexão

Note que em Sinal-Magnitude, para tornar um número negativo, basta "ligar" o bit mais significativo. O restante do número permanece idêntico.

O Esquema de Excesso de 2^{n-1}

Neste sistema, não "reservamos" um bit de sinal isolado. Em vez disso, somamos um valor fixo ao número real para deslocá-lo para a faixa positiva.

Funcionamento

Para n bits, o excesso é 2^{n-1} .

$$\text{Representação Binária} = \text{Valor Decimal} + 2^{n-1}$$

- **Inversão de Lógica:** Ao contrário do Sinal-Magnitude, aqui:
 - Números que começam com **0** são **Negativos**.
 - Números que começam com **1** são **Positivos** (ou zero).
- **Vantagem:** Mantém a **ordem crescente** dos bits igual à ordem dos valores decimais.
- **Zero Único:** O valor 0 é representado pelo padrão $100\dots 0_2$.

Exemplo: $n = 4$ (Excesso de $2^3 = 8$)

Com 4 bits, somamos 8 a cada valor para achar seu código:

Decimal	Cálculo (+8)	Binário
-8 (mínimo)	$-8 + 8 = 0$	0000
-7	$-7 + 8 = 1$	0001
0	$0 + 8 = 8$	1000
+1	$1 + 8 = 9$	1001
+7 (máximo)	$7 + 8 = 15$	1111

- **Faixa de Valores** ($n = 4$): de -8 a $+7$.
- **Regra Geral**: de $-(2^{n-1})$ a $2^{n-1} - 1$.

Tabela Completa: Excesso de $2^3 = 8$ ($n = 4$)

Binário	Cálculo	Decimal	Binário	Cálculo	Decimal
0000	$0 - 8$	-8	1000	$8 - 8$	0
0001	$1 - 8$	-7	1001	$9 - 8$	$+1$
0010	$2 - 8$	-6	1010	$10 - 8$	$+2$
0011	$3 - 8$	-5	1011	$11 - 8$	$+3$
0100	$4 - 8$	-4	1100	$12 - 8$	$+4$
0101	$5 - 8$	-3	1101	$13 - 8$	$+5$
0110	$6 - 8$	-2	1110	$14 - 8$	$+6$
0111	$7 - 8$	-1	1111	$15 - 8$	$+7$

- **MSB = 0:** Números negativos.
- **MSB = 1:** Números positivos e o zero.
- Esta representação é muito utilizada em expoentes de números em **Ponto Flutuante (IEEE 754)**.

Exercício 22: Excesso de 2^{n-1} ($n = 5$)

Para $n = 5$, o excesso é $2^{5-1} = 2^4 = \mathbf{16}$.

Regra: Some 16 ao número e converta o resultado para binário de 5 bits.

Converta para binário (5 bits, com Excesso de 2^{n-1}):

- a) -15
- b) -1
- c) 0
- d) 10

Gabarito: Exercício 22

a) $-15: -15 + 16 = 1 \rightarrow 00001_2$

b) $-1: -1 + 16 = 15 \rightarrow 01111_2$

c) $0: 0 + 16 = 16 \rightarrow 10000_2$

d) $10: 10 + 16 = 26 \rightarrow 11010_2$

Dica Visual

Repare que o zero (10000) divide a tabela exatamente no meio. Qualquer número menor que 16 começará com 0 (negativos), e qualquer número ≥ 16 começará com 1 (positivos).

Representação em Excesso de 16 ($n = 5$)

Para $n = 5$, o excesso é $2^{5-1} = 16$. A faixa de valores vai de -16 a $+15$.

Binário	Decimal	Binário	Decimal	Binário	Decimal
00000	-16	01011	-5	10110	+6
00001	-15	01100	-4	10111	+7
00010	-14	01101	-3	11000	+8
00011	-13	01110	-2	11001	+9
00100	-12	01111	-1	11010	+10
00101	-11	10000	0	11011	+11
00110	-10	10001	+1	11100	+12
00111	-9	10010	+2	11101	+13
01000	-8	10011	+3	11110	+14
01001	-7	10100	+4	11111	+15
01010	-6	10101	+5	—	

- **Cálculo:** Valor = Inteiro sem sinal $- 16$.
- O padrão 10000 sempre representa o zero real.

Aula 6 - Precisão Finita (parte 2)

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 9, 2026

- **Estratégia:** Inverter todos os bits para negar o número ($0 \rightarrow 1$ e $1 \rightarrow 0$).
- **Bit de Sinal:** O bit mais à esquerda define o sinal (0 pos, 1 neg).
- Assim como no Sinal-Magnitude, o Complemento a 1 possui **duas representações para o zero**:
 - 0000 (+0) e 1111 (−0 em $n = 4$).
- **Antes do Complemento a 2, utilizava-se o Complemento a 1**
- O **Complemento a 2** elimina o "zero duplo" e é o padrão nos computadores desde 1965.

Complemento a 1 com $n = 4$

Observe como cada número negativo é exatamente a inversão bit a bit do seu equivalente positivo:

Decimal	Binário	Inversão	Binário	Decimal
+0	0000	$\longleftrightarrow$	1111	-0
+1	0001	$\longleftrightarrow$	1110	-1
+2	0010	$\longleftrightarrow$	1101	-2
+3	0011	$\longleftrightarrow$	1100	-3
+4	0100	$\longleftrightarrow$	1011	-4
+5	0101	$\longleftrightarrow$	1010	-5
+6	0110	$\longleftrightarrow$	1001	-6
+7	0111	$\longleftrightarrow$	1000	-7

- **Funcionamento:** Para negar, basta aplicar o operador NOT (inverter 0 e 1).
- **O Problema:** Note o +0 (0000) e o -0 (1111). Essa duplicidade gera desperdício de uma combinação e circuitos de teste mais complexos.

Para encontrar o negativo de um número ($x \rightarrow -x$) em complemento a 2 faça o seguinte:

1. **Complemento a 1:** Inverta os bits.
2. **Somar 1:** Adicione 1 ao resultado obtido.

Exemplo de complemento a 2 ($+3 \rightarrow -3$):

- $+3 = 0011_2$
- Inverte (complemento a 1) $\rightarrow 1100_2$
- Some 1 $\rightarrow 1101_2$ (que é -3)

Complemento a 2: A Lógica dos Pesos

Neste sistema, o bit mais significativo (MSB) tem **peso negativo**, enquanto os demais mantêm pesos positivos:

Cálculo para $n = 4$ bits

- $0101 = -0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = \mathbf{5}$
- $1011 = -1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = -8 + 3 = \mathbf{-5}$
- $1111 = -1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = -8 + 7 = \mathbf{-1}$
- $1000 = -1 \cdot 2^3 + 0 + 0 + 0 = \mathbf{-8}$ (*Mínimo*)

Vantagem: O zero é único (0000) e as operações aritméticas são simplificadas no hardware.

Tabela Completa: Complemento a 2 ($n = 4$)

Binário	Valor Decimal	Binário	Valor Decimal
0000	0	1000	-8
0001	+1	1001	-7
0010	+2	1010	-6
0011	+3	1011	-5
0100	+4	1100	-4
0101	+5	1101	-3
0110	+6	1110	-2
0111	+7	1111	-1

Faixa de Valores: de -2^{n-1} até $2^{n-1} - 1$ (no caso, -8 a $+7$).

Visualizacao circular Complemento a 2

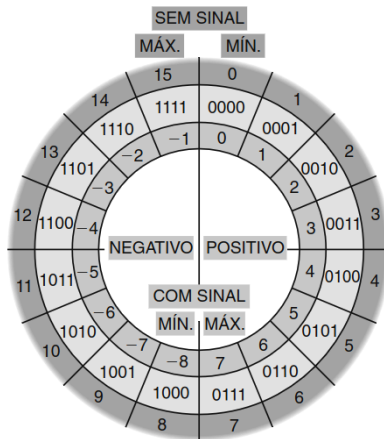


FIGURA 6.3 Um círculo numérico de quatro bits.

Números Positivos e o Zero em Complemento a 2

Para a faixa de 0 até $2^{n-1} - 1$, a representação em **Complemento a 2** é idêntica à de **Sinal-Magnitude**:

Regras de Representação

- **Bit de Sinal:** O bit mais significativo (MSB) é sempre **0**.
- **Magnitude:** Os demais bits seguem a notação posicional binária padrão.

Exemplo ($n = 4$)

O valor $+5_{10}$ é sempre 0101 em:

- Binário puro (4 bits)
- Sinal-Magnitude ($n = 4$)
- Complemento a 2 ($n = 4$)

Números Negativos em Complemento a 2

Para calcular a representação de um número negativo na faixa de -1 até -2^{n-1} , seguimos o algoritmo de três passos:

Algoritmo de Conversão ($x \rightarrow -x$)

1. **Magnitude:** Converte o valor absoluto do número para binário com n bits.
2. **Complemento a 1:** Inverte cada bit (0 vira 1, 1 vira 0).
3. **Soma 1** ao resultado anterior.
 - *Importante: Ignore o "vai-um" (carry-out) do bit mais significativo, se houver.*

Considere o descarte do carry: Este processo garante que a soma de um número com seu negativo resulte em zero.

Exemplos Práticos ($n = 4$)

Calculando o valor -3

- **Passo 1 ($|3|$):** 0011
- **Passo 2 (Comp. 1):** 1100
- **Passo 3 (Soma 1):** $1100 + 1 = 1101$
- **Resultado:** $-3_{10} = 1101_2$

O caso do Zero (0)

- **Passo 1 ($|0|$):** 0000
- **Passo 2 (Comp. 1):** 1111
- **Passo 3 (Soma 1):** $1111 + 1 = [1]0000$
- **Resultado:** 0000 (O "vai-um" é ignorado, mantendo o zero único).

Resumo de Comparação ($n = 4$)

Para o valor decimal 3:

- **Positivo (+3):** 0011
- **Negativo (-3):** 1101

$$\begin{array}{r} 0011 (+3) \\ + \quad 1101 (-3) \\ \hline [1] \quad 0000 (0) \end{array}$$

Dica de Hardware: Esse algoritmo é eficiente porque permite que o circuito de subtração seja basicamente um somador.

Faixa de Representação em Complemento a 2

Para uma palavra de n bits, a faixa de valores é assimétrica devido à representação única do zero:

- **Números Positivos e Zero:** De 0 até $2^{n-1} - 1$.
 - Temos $n - 1$ bits disponíveis para a magnitude.
- **Números Negativos:** De -1 até -2^{n-1} .
 - São 2^{n-1} combinações negativas.

Amplitude Total

A faixa completa vai de -2^{n-1} até $2^{n-1} - 1$.

- Total de valores: $2 \times 2^{n-1} = 2^n$ combinações distintas.
- **Vantagem:** Não há desperdício de combinação com o "zero duplo".

Exercício: Complemento a 2 com $n = 5$ bits

Escreva os números abaixo na base 2 usando a notação de **Complemento a 2**:

- a) -15
- b) -1
- c) 0
- d) 10

Respostas do Exercício ($n = 5$)

Confira os resultados para a representação de 5 bits:

Decimal	Binário (Complemento a 2)	Explicação
-15	10001	Inverte 01111 $\rightarrow$ 10000 +1
-1	11111	Inverte 00001 $\rightarrow$ 11110 +1
0	00000	Padrão único
10	01010	Positivo direto

Dica para os negativos: 1. Ache o positivo em 5 bits (ex:

15 = 01111).

2. Inverta os bits (10000).

3. Some 1 (10001).

Nota: Em $n = 5$, a faixa permitida é de -16 a $+15$. Todos os valores acima estão dentro do limite.

Exercício: Interpretando Complemento a 2 ($n = 7$)

Transcreva os números abaixo (em complemento a 2 de 7 bits) para a base 10.

a) 1111110

b) 1000001

c) 0111111

d) 1010101

Respostas do Exercício ($n = 7$)

Confira os valores correspondentes na base 10:

- a)** $1111110 \rightarrow \text{Inverte } (0000001) + 1 = 0000010_2 \rightarrow -2$
- b)** $1000001 \rightarrow \text{Inverte } (0111110) + 1 = 0111111_2 \rightarrow -63$
- c)** $0111111 \rightarrow \text{Começa com 0 (positivo)}$
 $\rightarrow 32 + 16 + 8 + 4 + 2 + 1 \rightarrow +63$
- d)** $1010101 \rightarrow \text{Inverte } (0101010) + 1 = 0101011_2 \rightarrow -43$

Se a sequência começa com 1, o número é negativo!

Negação (Inverso Aditivo) em Complemento a 2

"Negar" um número significa inverter o seu sinal ($x \rightarrow -x$).

- **No Sinal-Magnitude:** Basta inverter o bit mais significativo.
- **No Complemento a 2:** O processo de negação é **Complemento a 1 e Somar 1**.

Algoritmo

Independentemente de o número original ser positivo ou negativo, o processo para encontrar seu inverso aditivo é o mesmo:

1. Inverte-se todos os bits ($0 \leftrightarrow 1$).
2. Soma-se 1 ao resultado.

Exemplo ($n = 4$): De -3 para $+3$

- Partimos de -3 : 1101
- Invertemos os bits: 0010
- Somamos 1: $0010 + 1 = 0011$ (que é $+3$)

Exercício: Negação em 7 bits

Obtenha a representação da **negação** (inverso aditivo) de cada número abaixo, considerando a codificação Complemento a 2 com $n = 7$:

- a) 1111110
- b) 1000001
- c) 0111111
- d) 1010101

Lembre-se: O procedimento é o mesmo para todos: **Inverter + Somar 1.**

Respostas: Inverso Aditivo ($n = 7$)

a) 1111110 (-2)

- Inverte (0000001) $+1 \rightarrow$ **0000010** ($+2$)

b) 1000001 (-63)

- Inverte (0111110) $+1 \rightarrow$ **0111111** ($+63$)

c) 0111111 ($+63$)

- Inverte (1000000) $+1 \rightarrow$ **1000001** (-63)

d) 1010101 (-43)

- Inverte (0101010) $+1 \rightarrow$ **0101011** ($+43$)

Atenção

A negação de um número positivo **sempre** resultará em um número começando com 1, e vice-versa (exceto para o zero).

Adição em Complemento a 2

A grande vantagem do Complemento a 2 é que a adição é feita exatamente como nos números sem sinal, ignorando se o bit mais à esquerda representa sinal ou magnitude.

A Regra do Overflow

Diferente dos números sem sinal, o "vai-um" (carry-out) final não indica necessariamente um erro. No Complemento a 2, ocorre **overflow** apenas quando:

- Somamos dois números de **mesmo sinal** e o resultado tem **sinal oposto**.
- **Positivo + Negativo**: Nunca gera overflow.
- **Carry-out**: O vai-um que "sobra" à esquerda do bit de sinal deve ser **descartado**.

Exemplos de Adição ($n = 8$)

1. Positivos (sem overflow):

$$\begin{array}{r} \textcolor{red}{11} \text{ } \textcolor{red}{11} \\ 00011011_2 (27) \\ + 01001001_2 (73) \\ \hline 01100100_2 (100) \end{array}$$

2. Negativos (sem overflow):

$$\begin{array}{r} \textcolor{red}{11111} \text{ } \textcolor{red}{11} \\ 11111011_2 (-5) \\ + 11001001_2 (-55) \\ \hline [1]11000100_2 (-60) \end{array}$$

3. Sinais opostos:

$$\begin{array}{r} \textcolor{red}{11} \text{ } \textcolor{red}{11} \\ 11001001_2 (-55) \\ + 00011011_2 (27) \\ \hline 11100100_2 (-28) \end{array}$$

4. Negativos com Overflow:

$$\begin{array}{r} \textcolor{red}{1} \text{ } \textcolor{red}{11} \text{ } \textcolor{red}{11} \\ 10011011_2 (-99) \\ + 11001001_2 (-60) \\ \hline [1]01100100_2 !! \end{array}$$

Erro: Negativo + Negativo deu Positivo.

Subtração em Complemento a 2

A subtração é simplificada transformando-a em uma adição com o inverso aditivo:

$$A - B \implies A + (-B)$$

Procedimento

1. Mantenha o primeiro operando (*op1*).
2. Calcule o **Complemento a 2** do segundo operando (*op2*).
3. Some os dois valores seguindo as regras da adição.

Isso permite que o hardware use o mesmo circuito somador para as duas operações.

Exercício: Aritmética com 4 bits

Realize as operações usando **Complemento a 2 com 4 bits**. Indique se houve overflow.

a) $5 - 3$

b) $2 - 4$

c) $(-5) + (-1)$

d) $4 + 2$

d) $7 + 3$

d) $(-4) + (-5)$

Lembre-se da faixa de 4 bits: $[-8, +7]$.

Gabarito: Aritmética com 4 bits

- a) $0101 + 1101 = [1]0010$ (+2) OK
- b) $0010 + 1100 = 1110$ (-2) OK
- c) $1011 + 1111 = [1]1010$ (-6) OK
- d) $0100 + 0010 = 0110$ (+6) OK
- e) $0111 + 0011 = \mathbf{1010}$ OVERFLOW!
- *Positivo + Positivo resultou em negativo.*
- f) $1100 + 1011 = [1]\mathbf{0111}$ OVERFLOW!
- *Negativo + Negativo resultou em positivo.*

Multiplicação e Divisão em Complemento a 2

Diferente da adição, a multiplicação e a divisão no Complemento a 2 geralmente seguem uma estratégia de "valor absoluto":

Algoritmo Geral

1. **Preparação:** Se um operando for negativo, calcula-se seu Complemento a 2 para obter o valor absoluto ($|x|$).
2. **Operação:** Multiplica-se ou divide-se os valores absolutos como números sem sinal.
3. **Sinal do Resultado:**
 - Sinais iguais $\rightarrow$ Resultado Positivo.
 - Sinais opostos $\rightarrow$ Calcula-se o Complemento a 2 do resultado para torná-lo negativo.

Overflow na Multiplicação

Em uma representação de n bits, ocorre overflow se o produto dos valores absolutos atingir o bit de sinal ($n - 1$) ou além.

Exemplos de Multiplicação ($n = 8$)

1. Positivos com Overflow: $27 \times 5 = 135$ (Limite é 127)

	0	0	0	1	1	0	1	1_2
$\times$	0	0	0	0	0	1	0	1_2
	1!!	1	1	1				
	0	0	0	1	1	0	1	1
+	0	1	1	0	1	1		
	1!!	0	0	0	0	1	1	1

2. Positivos sem Overflow: $11 \times 5 = 55$

	0	0	0	0	1	0	1	1_2
$\times$	0	0	0	0	0	1	0	1_2
				1				
	0	0	0	0	1	0	1	1
+	0	0	1	0	1	1		
	0	0	1	1	0	1	1	1

Exemplo: Multip. de positivo e um negativo com *overflow*

Considere: 10001011_2 (-117) $\times$ 00000101_2 (5) em $n = 8$.

Passo 1: Obter valor absoluto de -117 :

	1	0	0	0	1	0	1	1_2
inv	0	1	1	1	0	1	0	0_2
+								1
	0	1	1	1	0	1	0	1_2

Passo 2: Multiplicar 117×5 :

	0	1	1	1	0	1	0	1_2
$\times$	0	0	0	0	0	1	0	1_2
	1	1	1	1		1		
	0	1	1	1	0	1	0	1
+ 1!!	1!!	1	0	1	0	1		
... !!	0	1	0	0	1	0	0	1

Expansão de Bits (Extensão de Sinal)

Frequentemente, precisamos converter um número de n bits para uma representação de m bits ($m > n$) mantendo seu valor original.

A Regra da Extensão de Sinal

Para expandir um número em **Complemento a 2**:

1. Move-se o bit de sinal para a nova posição mais à esquerda.
 2. Preenchem-se as novas posições com o **mesmo valor do bit de sinal**.
- Se o número é **positivo** (MSB=0), preenchemos com **0s** à esquerda.
 - Se o número é **negativo** (MSB=1), preenchemos com **1s** à esquerda.

Exemplos de Expansão ($4 \rightarrow 8$ bits)

Exemplo 1: Valor Positivo (+4)

- Representação 4 bits: 0100
- Extensão para 8 bits: 00000100

Exemplo 2: Valor Negativo (-4)

- Representação 4 bits: 1100
- Extensão para 8 bits: 11111100

Por que funciona? No caso do -4:

- 4 bits: $-8 + 4 = -4$
- 8 bits: $-128 + 64 + 32 + 16 + 8 + 4 = -128 + 124 = -4$

Exercício: Extensão de Sinal e Hexadecimal

Mostre que os valores hexadecimais abaixo, em notação de **Complemento a 2**, representam o valor -5_{10} :

- a) B ($n = 4$ bits)
- b) FB ($n = 8$ bits)
- c) FFB ($n = 12$ bits)

Dica de Resolução

Converta o dígito mais significativo para binário e verifique o bit de sinal.

- B em binário é 1011.
- Como o MSB é **1**, o número é negativo.
- Aplique o processo inverso (Inverter + Somar 1) para conferir a magnitude.

Resolução: A consistência do -5

a) $B \rightarrow 1011$.

- Inverso: $0100 + 1 = 0101$ (5_{10}). Logo: -5 .

b) $FB \rightarrow 1111\ 1011$.

- Inverso: $0000\ 0100 + 1 = 0000\ 0101$ (5_{10}). Logo: -5 .

c) $FFB \rightarrow 1111\ 1111\ 1011$.

- Inverso: $0000\ 0000\ 0100 + 1 = 0000\ 0101$ (5_{10}). Logo: -5 .

Conclusão

Note que FB e FFB são apenas extensões de sinal de B. Em hexadecimal, se o número for negativo, ele será preenchido com F (1111_2) à esquerda.

Redução e Truncamento

O caminho inverso (reduzir o número de bits) nem sempre preserva o valor. Ao truncar de n para k bits ($k < n$), eliminamos os $n - k$ bits mais significativos.

- **Números sem sinal:** O truncamento equivale a $x \bmod 2^k$.

Exemplo: Truncando 1100_2 (12_{10})

- **Para 3 bits:** $1|100 \rightarrow 100_2$ (4_{10}).
 - Cálculo: $12 \bmod 2^3 = 12 \bmod 8 = 4$.
- **Para 2 bits:** $11|00 \rightarrow 00_2$ (0_{10}).
 - Cálculo: $12 \bmod 2^2 = 12 \bmod 4 = 0$.

Atenção: No Complemento a 2, o truncamento pode alterar drasticamente o valor e até o **sinal** do número!

Aula 6 - Precisão Finita (parte 3)

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 9, 2026

Notação Posicional Fracionária

A representação de números fracionários expande a notação posicional para a direita do separador decimal, utilizando potências negativas da base B .

$$\left(\sum_{i=0}^m b_i B^i\right) + \left(\sum_{j=-n}^{-1} b_j B^j\right)$$

Estrutura de Pesos

2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}
4	2	1	.	0,5	0,25	0,125

Exemplo: Binário $\rightarrow$ Decimal

$$10100.011_2 = (16 + 4) + (0,25 + 0,125) = 20,375$$

Conversão Decimal $\rightarrow$ Binário

Para converter um número fracionário da base 10 para a base 2:

1. Converta a **parte inteira** normalmente (divisões sucessivas).
2. Use o algoritmo de **multiplicações sucessivas** para a parte fracionária (n).

Algoritmo de Multiplicação

1. Multiplique n por 2.
2. A parte inteira do resultado (0 ou 1) é o próximo bit.
3. A nova parte fracionária vira o novo n .
4. Repita até $n = 0$ ou atingir a precisão desejada.

Exemplo: Conversão de 20,375

Parte inteira: $20_{10} = 10100_2$. Parte fracionária: 0,375

- $0,375 \times 2 = \mathbf{0},75 \rightarrow b_{-1} = 0$
- $0,75 \times 2 = \mathbf{1},5 \rightarrow b_{-2} = 1$
- $0,5 \times 2 = \mathbf{1},0 \rightarrow b_{-3} = 1$ (Fim, pois $n = 0$)

Resultado: $10100,011_2$

O Problema da Dízima Binária

Nem todo número decimal exato possui uma representação binária exata.

Exemplo: 20,3

- $0,3 \times 2 = 0,6 \rightarrow 0$
- $0,6 \times 2 = 1,2 \rightarrow 1$
- $0,2 \times 2 = 0,4 \rightarrow 0$
- $0,4 \times 2 = 0,8 \rightarrow 0$
- $0,8 \times 2 = 1,6 \rightarrow 1$
- $0,6 \times 2 = 1,2 \rightarrow 1$ (Ciclo iniciado!)

Converta os seguintes números da base 10 para a base 2:

- a) 20,5
- b) 7,25
- c) 0,625
- d) 1,1

Gabarito: Conversão Fracionária

a) 20,5:

- Parte inteira: $20 = 10100_2$
- Parte fracionária: $0,5 \times 2 = 1,0$
- **Resultado:** $10100,1_2$

b) 7,25:

- Parte inteira: $7 = 111_2$
- Parte fracionária: $0,25 \times 2 = 0,5 \rightarrow 0,5 \times 2 = 1,0$
- **Resultado:** $111,01_2$

c) 0,625:

- Parte inteira: $0 = 0_2$
- Parte fracionária:
 $0,625 \times 2 = 1,25 \rightarrow 0,25 \times 2 = 0,5 \rightarrow 0,5 \times 2 = 1,0$
- **Resultado:** $0,101_2$

d) 1,1:

- Parte inteira: $1 = 1_2$
- Parte fracionária: $0,1 \times 2 = 0,2 \rightarrow 0,2 \times 2 = 0,4 \rightarrow 0,4 \times 2 = 0,8 \rightarrow 0,8 \times 2 = 1,6 \rightarrow 0,6 \times 2 = 1,2 \dots$
- **Resultado:** $1,00011(0011) \dots_2$ (Dízima periódica)

O objetivo deste padrão é a interoperabilidade: garantir que um número fracionário gerado em um sistema seja interpretado da mesma forma em qualquer outro.

- **Categorias de Números:** Define representações para números normalizados, não normalizados e valores especiais.
- **Valores Especiais:**
 - Zero ($+0$ e -0)
 - Infinito (∞ e $-\infty$)
 - **NaN** (*Not a Number*): Resultados de operações inválidas (ex: $0/0$).
- **Formatos Comuns:**
 - **Precisão Simples (float):** 32 bits.
 - **Precisão Dupla (double):** 64 bits.

Notação Científica Binária

A base do padrão é a normalização do número binário, similar à notação científica decimal ($1,2 \times 10^3$). No binário, qualquer número (exceto o zero) pode ser escrito como:

$$\pm 1,xxxxx_2 \times 2^{yyyy}$$

Componentes da Representação

- **Sinal (S):** Indica se o número é positivo ou negativo.
- **Mantissa ou Significando (M):** A parte fracionária (xxxxx).
 - *Nota: O "1" antes da vírgula é implícito (bit oculto) para economizar espaço.*
- **Expoente (E):** O valor da potência de 2 (yyyy).

Layout da Precisão Simples (32 bits)

A representação de 32 bits é dividida em três campos específicos:

- **Sinal (S):** 1 bit (índice 31).
- **Expoente (*exp*):** 8 bits (índices 30 a 23).
- **Mantissa (*frac*):** 23 bits (índices 22 a 0).

Arrumação dos Bits

S **EEEEEEEE** **MMMMMMMMMMMMMMMMMMMMMMMM**

Dica de Leitura

Como 32 bits são difíceis de ler, agrupamos de 4 em 4 e representamos como **8 dígitos hexadecimais**.

O Expoente

O expoente real (E) não é armazenado diretamente em Complemento a 2. Usa-se um **a lógica do excesso** de 127.

$$exp = E + 127$$

- **Faixa do Expoente Real (E):** -126 a $+127$.
- **Faixa no Campo (exp):** 1 a 254.

Significado	Expoente Real (E)	Valor em Bits (exp)
Menor valor	-126	00000001_2 (1)
Zero real	0	01111111_2 (127)
Maior valor	127	11111110_2 (254)

**As combinações 00000000 e 11111111 são reservadas para casos especiais (Zero, NaN, Infinito).*

A Mantissa e o Bit Oculto

Na notação normalizada, o número sempre começa com $1.xxxxx_2$.

Economia de Bits

Como o "1" antes da vírgula é sempre fixo para números normalizados, o padrão IEEE 754 **não o armazena**. Guardamos apenas a parte fracionária nos 23 bits.

Pesos da Mantissa (*frac*): Os bits da esquerda para a direita representam potências negativas:

$$2^{-1}, 2^{-2}, 2^{-3}, \dots, 2^{-23}$$

Exemplo: Decodificando o Expoente

Se o campo de expoente na memória for 11110000:

1. **Converter para Decimal (*exp*):**

$$11110000_2 = 128 + 64 + 32 + 16 = \mathbf{240}$$

2. **Subtrair o Viés (127):** $E = exp - 127$ $E = 240 - 127 = \mathbf{113}$

O número real está sendo multiplicado por 2^{113} .

Exemplo 1: Decodificação do Menor Expoente

Imagine que os 8 bits de expoente em uma representação IEEE 754 (Precisão Simples) sejam 00000001.

Passo 1: Decodificar o campo exp (Sem sinal)

$$exp = 00000001_2 = 2^0 = 1$$

Passo 2: Calcular o expoente real E

Utilizando a lógica do excesso de 127 ($exp = E + 127$):

$$E = exp - 127$$

$$E = 1 - 127 = -\mathbf{126}$$

Este é o menor expoente real possível de ser representado em precisão simples para números normalizados.

Exemplo 2: Decodificação de Expoente Intermediário

Imagine agora que os 8 bits do campo de expoente sejam 11110000.

Passo 1: Decodificar o campo exp

$$exp = 11110000_2 = 2^7 + 2^6 + 2^5 + 2^4$$

$$exp = 128 + 64 + 32 + 16 = \mathbf{240}$$

Passo 2: Calcular o expoente real E

$$E = exp - 127$$

$$E = 240 - 127 = \mathbf{113}$$

Portanto, a potência de base 2 que multiplica a mantissa neste caso é 2^{113} .

Exemplo 3: Codificação do Maior Expoente

Agora queremos representar o maior expoente real possível em precisão simples: $E = 127$. Como codificar isso nos bits 30 a 23?

Passo 1: Obter o valor do campo exp (Soma do excesso)

$$exp = E + 127$$

$$exp = 127 + 127 = \mathbf{254}$$

Passo 2: Converter exp para binário (8 bits)

Convertendo 254 para a base 2 (sem sinal):

$$254 = 11111110_2$$

Esta é a sequência de bits que deve ser inserida no campo de expoente para representar $E = 127$.

A Parte Fracionária da Mantissa

Os 23 bits reservados para a parte fracionária usam uma codificação posicional sem sinal.

- O bit mais à esquerda corresponde ao expoente -1 .
- O segundo bit ao expoente -2 , e assim por diante.
- O mapeamento completo pode ser visto.

posição	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
potência de 2	-1	-2	-3	-4	-5	-6	-7	-8	-9	-10	-11	-12	-13	-14	-15	-16	-17	-18	-19	-20	-21	-22	-23
coeficiente (bit)	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1

Figure: Mapeamento dos bits da mantissa e potências de 2.

Pesos das Potências Negativas de 2

Potência	Fração	Decimal
2^{-1}	$1/2$	0,5
2^{-2}	$1/4$	0,25
2^{-3}	$1/8$	0,125
2^{-4}	$1/16$	0,0625
2^{-5}	$1/32$	0,03125
2^{-6}	$1/64$	0,015625

Table: Valores decimais das potências negativas de 2.

Exemplo: Mantissa 1,25 (Parte frac = 0,25)

Se a mantissa (base 10) é 1,25, sua parte fracionária é 0,25.

Primeiro, convertemos para binário: $0,25_{10} = 0,01_2$.

- Preenchemos os bits iniciais e completamos com zeros à direita.
- Podemos incluir tantos "zeros à direita" quanto necessários para preencher os 23 bits.

posição	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
potência de 2	-1	-2	-3	-4	-5	-6	-7	-8	-9	-10	-11	-12	-13	-14	-15	-16	-17	-18	-19	-20	-21	-22	-23
coeficiente (bit)	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure: Preenchimento dos bits para a parte fracionária 0,25.

Decodificando a Mantissa

Para decodificar uma representação IEEE 754, o processo é o inverso:

1. Identificamos a sequência de bits fracionários.
2. Acrescentamos o **1 à esquerda** (bit oculto).
3. Convertemos o número binário resultante para a base 10.

Exemplo Prático

Dada a sequência: 110100000000000000000000

- Mantissa em binário: $1,1101_2$
- Conversão: $M = 1 + 2^{-1} + 2^{-2} + 2^{-4}$
- $M = 1 + 0,5 + 0,25 + 0,0625 = \mathbf{1,8125}$

Codificação e Decodificação Completa

Agora que já sabemos codificar e decodificar cada trecho dos 32 bits do padrão, podemos juntar as informações para codificar ou decodificar o número sendo representado.

Exemplo 1: Decodificando um float

Decodifique a sequência: 0 10000000 111000000000000000000000

Codificação e Decodificação Completa

Agora que já sabemos codificar e decodificar cada trecho dos 32 bits do padrão, podemos juntar as informações para codificar ou decodificar o número sendo representado.

Exemplo 1: Decodificando um float

Decodifique a sequência: 0 10000000 111000000000000000000000

- **Sinal:** $S = 0 \implies$ Positivo.
- **Expoente:** $exp = 10000000_2 = 128_{10}$.
- **Expoente Real:** $E = 128 - 127 = 1$.
- **Mantissa:** $frac = 111... \implies M = 1 + 2^{-1} + 2^{-2} + 2^{-3}$
- $M = 1 + 0,5 + 0,25 + 0,125 = \mathbf{1,875}$.
- **Resultado:** $1,875 \times 2^1 = \mathbf{3,75}$.

Agora que já sabemos decodificar cada trecho, podemos juntar as informações para encontrar o valor real do número.

Deslocamento da Vírgula

Como $E = 1$, podemos deslocar a vírgula binária **uma casa para a direita** antes de converter para decimal:

$$1,111_2 \times 2^1 = \mathbf{11,11_2}$$

Conversão Final:

$$11,11_2 = 2^1 + 2^0 + 2^{-1} + 2^{-2} = 2 + 1 + 0,5 + 0,25 = \mathbf{3,75}$$

Exemplo 2: Construindo a representação (1/2)

Vamos construir a representação float do número **−4535,8125**.

- **Passo 1** : $S = 1$ (negativo).
- **Passo 2** : Convertendo a parte inteira (Fig. 3) e a fracionária (Fig. 4):

$$4535,8125_{10} = 1000110110111,1101_2$$

dividendo	quociente	resto	posição
4535	2267	1	b0
2267	1133	1	b1
1133	566	1	b2
566	283	0	b3
283	141	1	b4
141	70	1	b5
70	35	0	b6
35	17	1	b7
17	8	1	b8
8	4	0	b9
4	2	0	b10
2	1	0	b11
1	0	1	b12

Figure: Conversão de 4535.

número	dobro	parte inteira	posição
0.8125	1.625	1	b-1
0.625	1.25	1	b-2
0.25	0.5	0	b-3
0.5	1	1	b-4
0	0		

Figure: Conversão de 0,8125.

Exemplo 2: Construindo a representação (2/2)

- **Passo 3** : Mover a vírgula para obter a forma $1,xxxx \times 2^E$:

$$1000110110111,1101 = 1,0001101101111101 \times 2^{12}$$

- **Passo 4** :

- $M = 1,0001101101111101 \implies \text{frac} = \mathbf{0001101101111101}$
- $E = 12 \implies \text{exp} = 12 + 127 = 139$
- 139 em binário de 8 bits: **10001011**

- **Passo 5** : 1 | 10001011 | 00011011011111010000000

Representação Final

Agrupando de 4 em 4 bits para Hexadecimal: 1100 0101 1000 1101
1011 1110 1000 0000

0xC58DBE80

Precisão Dupla (64 bits)

O padrão IEEE 754 também define a precisão dupla, que utiliza o dobro de bits para aumentar tanto a faixa de valores quanto a precisão.

- **Sinal (S):** 1 bit (índice 63).
- **Expoente (*exp*):** 11 bits (índices 62 a 52).
 - Lógica do excesso: $exp = E + 1023$.
- **Mantissa (*frac*):** 52 bits (índices 51 a 0).

Arrumação dos Bits

S EEEEEEEEEEE MMMMMMMMMMMMMMMMMMM... (52x)

Curiosidade

Enquanto o *float* (simples) tem cerca de 7 dígitos decimais de precisão, o *double* (dupla) oferece cerca de **15 a 17 dígitos**.

Exemplo: O número 2 em Precisão Dupla

Como representar o número inteiro 2_{10} em 64 bits?

- **Sinal:** $S = 0$ (positivo).
- **Normalização:** $2_{10} = 10_2 = 1,0 \times 2^1$.
- **Expoente:** $E = 1 \implies exp = 1 + 1023 = 1024$.
 - 1024 em 11 bits: 10000000000_2 .
- **Mantissa (*frac*):** Apenas zeros (52 bits).

Resultado em Hexadecimal (8 bytes)

Agrupando os 64 bits:

40 00 00 00 00 00 00 00

Desafio para a turma

Você consegue imaginar uma regra sobre a parte fracionária da mantissa na representação IEEE 754 de números inteiros que são **potências de 2**?

Dica: Pense no que acontece com a vírgula na normalização de números como 1, 2, 4, 8, 16, ...

- $1 = 1,0 \times 2^0$
- $2 = 1,0 \times 2^1$
- $4 = 1,0 \times 2^2$
- $8 = 1,0 \times 2^3$